

Documentazione Architettura e Tech Stack - KARIS

Tommaso Caputi, Di Pasquale Giulio, Gadaleta Giovanni

29 dicembre 2025

Indice

1 Introduzione

KARIS è un'applicazione web full-stack progettata per la gestione digitale delle risorse e della distribuzione di beni nelle strutture Caritas parrocchiali. Il sistema permette di gestire inventari, assegnazioni, beneficiari e facilita l'interscambio cooperativo tra diverse parrocchie.

Questa documentazione descrive l'architettura tecnica del sistema, le tecnologie utilizzate, come i componenti interagiscono tra loro e presenta un caso d'uso completo con il relativo workflow.

2 Tech Stack

2.1 Frontend

Tecnologia	Versione	Utilizzo
Next.js	16.0.10	Framework React per SSR/SSG e routing
React	19.2.1	Libreria UI per componenti interattivi
TypeScript	5.x	Linguaggio per type safety
Tailwind CSS	4.x	Framework CSS utility-first
Radix UI	Latest	Componenti UI accessibili e headless
Lucide React	0.561.0	Libreria di icone
Sonner	2.0.7	Sistema di notifiche toast

2.2 Backend

Tecnologia	Versione	Utilizzo
Next.js API Routes	16.0.10	Endpoint REST per logica server-side
Supabase JS Client	2.49.1	Client per database PostgreSQL e autenticazione

2.3 Database e Infrastruttura

Tecnologia	Utilizzo
PostgreSQL	Database relazionale (gestito via Supabase)
Supabase	BaaS (Backend as a Service) per database e auth
Supabase Auth	Sistema di autenticazione e gestione utenti

2.4 Strumenti di Sviluppo

Tecnologia	Utilizzo
ESLint	Linter per qualità del codice
PostCSS	Processore CSS
TypeScript Compiler	Compilazione e type checking

3 Architettura del Sistema

3.1 Architettura Generale

KARIS segue un'architettura **full-stack monolitica** basata su Next.js, che integra frontend e backend in un'unica applicazione. Il sistema utilizza il pattern **App Router** di Next.js 13+ per il routing e la gestione delle pagine.

Il sistema è organizzato in più livelli:

1. **Browser Client:** Componenti React renderizzati nel browser
2. **Next.js App Router:** Server Components per routing e rendering
3. **Next.js API Routes:** Endpoint REST per logica backend
4. **Supabase Client:** Interfaccia per database e autenticazione
5. **Supabase Cloud:** Database PostgreSQL e servizi di autenticazione

3.2 Pattern Architeturali

1. **Server-Side Rendering (SSR):** Le pagine vengono renderizzate sul server per migliorare SEO e performance iniziale
2. **Client-Side Rendering (CSR):** Componenti interattivi renderizzati lato client
3. **API Routes:** Endpoint REST per operazioni CRUD e logica di business
4. **Component-Based Architecture:** UI costruita con componenti React riutilizzabili
5. **Separation of Concerns:** Separazione tra logica di presentazione, business e data access

4 Struttura del Progetto

La struttura del progetto è organizzata come segue:

```

karis/
+-- db/                                # Script SQL per database
|   +-- schema.sql                    # Schema completo del database
|   +-- seed.sql                      # Dati di esempio
|
+-- documentazione/                   # Documentazione del progetto
|   +-- DOCUMENTAZIONE_DB.md
|   +-- DOCUMENTAZIONE_ARCHITETTURA.md
|   +-- er_schema.png
|
+-- karis/                             # Applicazione Next.js
    +-- src/
        |   +-- app/                  # App Router (Next.js 13+)
        |   |   +-- api/              # API Routes (Backend)
        |   |   |   +-- assegnazioni/
        |   |   |   +-- beneficiario/
        |   |   |   +-- beni/
        |   |   |   +-- dashboard/
        |   |   |   +-- famiglia/
        |   |   |   +-- inviti/
        |   |   |   +-- richieste/
        |   |   |   +-- user/
        |   |   |   +-- volontari/
        |   |   |
        |   |   +-- beneficiario/    # Pagine per gestione beneficiari
        |   |   +-- beni/            # Pagine per gestione beni
        |   |   +-- dashboard/       # Dashboard principale
        |   |   +-- famiglia/        # Pagine per gestione famiglie

```

```

| |   +--- invito/           # Pagina per accettazione inviti
| |   +--- login/           # Pagina di login
| |   +--- richieste/       # Pagina per richieste tra parrocchie
| |   +--- volontari/       # Pagina per gestione volontari
| |   +--- layout.tsx       # Layout root dell'applicazione
| |   +--- page.tsx         # Homepage (landing page)
| |   +--- globals.css      # Stili globali
| |
| +--- components/         # Componenti React riutilizzabili
| |   +--- ui/             # Componenti UI base (shadcn/ui)
| |       +--- button.tsx
| |       +--- dialog.tsx
| |       +--- input.tsx
| |       +--- ...
| |   +--- DashboardLayout.tsx # Layout condiviso per dashboard
| |   +--- LandingPage/       # Componenti per landing page
| |   +--- NotFound.tsx
| |
| +--- lib/                # Utilities e helper
| |   +--- supabaseClient.ts # Client Supabase configurato
| |   +--- apiHelpers.ts     # Helper per API routes
| |   +--- utils.ts          # Utility generiche
| |
+--- public/               # File statici (immagini, favicon, etc.)
+--- package.json
+--- tsconfig.json
+--- next.config.ts
+--- tailwind.config.js

```

5 Componenti e Interazioni

5.1 Livello 1: Componenti UI (Frontend)

5.1.1 DashboardLayout

Posizione: src/components/DashboardLayout.tsx

Responsabilità:

- Layout condiviso per tutte le pagine autenticate
- Sidebar di navigazione
- Header con informazioni utente
- Gestione autenticazione e logout

Interazioni:

- Utilizza `supabase.auth.getUser()` per verificare autenticazione
- Chiama `/api/user` per recuperare dati utente
- Gestisce navigazione tra sezioni

5.1.2 Componenti di Pagina

Ogni pagina (es. `beni/page.tsx`, `beneficiario/page.tsx`) è un componente React che:

- Utilizza `DashboardLayout` come wrapper
- Effettua chiamate API per recuperare dati
- Gestisce stato locale con React hooks (`useState`, `useEffect`)
- Mostra feedback all'utente tramite toast notifications (Sonner)

5.2 Livello 2: API Routes (Backend)

5.2.1 Struttura API Route

Ogni API route segue questo pattern:

```
1 // src/app/api/[resource]/route.ts
2 export async function GET(request: Request) {
3     // 1. Validazione parametri
4     // 2. Autenticazione utente
5     // 3. Recupero dati da database
6     // 4. Formattazione risposta
7     // 5. Return JSON
8 }
9
10 export async function POST(request: Request) {
11     // 1. Validazione body
12     // 2. Autenticazione utente
13     // 3. Validazione business logic
14     // 4. Inserimento/aggiornamento database
15     // 5. Return risultato
16 }
```

5.2.2 Helper Functions

Posizione: `src/lib/apiHelpers.ts`

Funzioni riutilizzabili per API routes:

- `getUserParrocchia(userId)`: Recupera utente e parrocchia associata
- `validateUserId()`: Valida presenza `userId` nei parametri
- `normalizeSupabaseRelation()`: Normalizza relazioni Supabase

5.3 Livello 3: Database Layer

5.3.1 Supabase Client

Posizione: `src/lib/supabaseClient.ts`

Configurazione del client Supabase:

- Inizializzazione con URL e API key da variabili d'ambiente
- Gestione errori quando Supabase non è configurato
- Utilizzato sia lato client che server

5.3.2 Query Pattern

Le query seguono questo pattern:

```
1 const { data, error } = await supabase
2   .from("tabella")
3   .select("campo1, campo2, relazione:campo_id (id, nome)")
4   .eq("campo", valore)
5   .order("created_at", { ascending: false });
```

5.4 Flusso di Dati Completo

Il flusso di dati segue questa sequenza:

1. **User Action:** L'utente interagisce con il componente (click, submit)
2. **React Component:** Gestisce lo stato e prepara la richiesta
3. **API Call:** Chiamata `fetch('/api/...')` al server
4. **API Route:** Validazione, autenticazione e business logic
5. **Supabase Client:** Query builder per database
6. **Database:** Operazioni su PostgreSQL
7. **Response:** Ritorno dati formattati in JSON
8. **UI Update:** Aggiornamento stato e re-render

6 Flusso di Autenticazione

6.1 Login

Il processo di login segue questi passaggi:

1. L'utente inserisce email e password nel form di login
2. Il componente `/login/page.tsx` gestisce lo stato del form
3. Chiamata a `supabase.auth.signInWithPassword()`
4. Supabase Auth verifica le credenziali e genera un session token
5. Il client salva la session in `localStorage`
6. Redirect automatico alla dashboard

6.2 Verifica Autenticazione

Ogni pagina protetta verifica l'autenticazione:

```
1 // Pattern comune in tutte le pagine
2 useEffect(() => {
3   const checkAuth = async () => {
4     const { data: { user }, error } = await supabase.auth.getUser();
5     if (error || !user) {
6       router.push("/login");
7     }
8   };
9   checkAuth();
10 }, []);
```

6.3 Recupero Dati Utente

Dopo l'autenticazione, il sistema recupera i dati dell'utente dal database:

```
1 // Chiamata API
2 const res = await fetch(`/api/user?userId=${user.id}`);
3 const userData = await res.json();
4 // userData contiene: nome, cognome, parrocchia, ruolo
```

6.4 Autorizzazione

Le API routes verificano che l'utente abbia accesso alle risorse della propria parrocchia:

```
1 // Pattern in API routes
2 const userResult = await getUserParrocchia(userId);
3 const parrocchiaId = userResult.data.parrocchia_id;
4
5 // Filtra query per parrocchia
6 query = query.eq("parrocchia_id", parrocchiaId);
```

7 Caso d'Uso: Assegnazione di un Bene

7.1 Scenario

Un volontario della Parrocchia di San Paolo vuole assegnare 5 scatolette di tonno a un beneficiario registrato nel sistema.

7.2 Workflow Completo

7.2.1 Fase 1: Accesso alla Funzionalità

1. Il volontario naviga a /beni
2. La pagina mostra la lista dei beni disponibili con pulsante "Assegna" per ogni bene
3. Click su "Assegna" per il tonno
4. Navigazione a /beni/assegna?beneId=...
5. Caricamento dati iniziali (beni, beneficiari, famiglie)

Codice rilevante:

```
1 // /beni/assegna/page.tsx
2 useEffect(() => {
3   const loadData = async () => {
4     // 1. Verifica autenticazione
5     const { data: { user } } = await supabase.auth.getUser();
6
7     // 2. Carica beni disponibili
8     const beniRes = await fetch(`/api/beni?userId=${user.id}`);
9     const beni = await beniRes.json();
10
11    // 3. Carica beneficiari
12    const beneficiariRes = await fetch(`/api/beneficiario?userId=${user.id}`);
13    const beneficiari = await beneficiariRes.json();
14
15    // 4. Carica famiglie
16    const famiglieRes = await fetch(`/api/famiglia?userId=${user.id}&all=true`);
```



```
17     const famiglie = await famiglieRes.json();
18   };
19   loadData();
20 }, []);
```

7.2.2 Fase 2: Selezione e Compilazione Form

Il form di assegnazione richiede:

1. Selezione Bene: Tonno (già pre-selezionato)
2. Tipo: Beneficiario o Famiglia (utente seleziona)
3. Beneficiario: Mario Rossi (utente seleziona)
4. Quantità: 5 (utente inserisce)
5. Note: (opzionale)

Validazione Client-Side:

```
1 // Verifica quantita disponibile
2 const beneSelezionato = beni.find(b => b.id === formData.beneId);
3 if (quantitaNum > beneSelezionato.quantity) {
4   toast.error(`Quantita non disponibile. Disponibile: ${beneSelezionato.quantity}`);
5   return;
6 }
```

7.2.3 Fase 3: Invio Richiesta

Quando l'utente clicca "Assegna Bene":

1. Validazione del form
2. Preparazione del payload
3. Invio POST a /api/assegnazioni

7.2.4 Fase 4: Elaborazione Backend

L'API route /api/assegnazioni/route.ts (POST) esegue:

1. Validazione Parametri:

- risorsa_id presente?
- quantita > 0?
- beneficiario_id o famiglia_id presente?

2. Autenticazione:

- getUserParrocchia(userId)
- Recupera parrocchia_id utente

3. Verifica Risorsa:

- Risorsa esiste?
- Risorsa appartiene alla parrocchia?

4. Verifica Beneficiario:

- Beneficiario esiste?
- Beneficiario appartiene alla parrocchia?

5. Controllo Disponibilità:

- Recupera inventario_parrocchia
- Calcola quantità già assegnata
- Verifica: $\text{quantita} \leq \text{disponibile}$

6. Creazione Assegnazione:

- Insert in assegnazione_bene
- Dati: risorsa_id, beneficiario_id, quantita, note

7. Risposta:

- Return JSON con assegnazione creata

Codice rilevante:

```
1 // /api/assegnazioni/route.ts
2 export async function POST(request: Request) {
3   // 1. Parse body
4   const { risorsa_id, beneficiario_id, quantita, note } = await request.json();
5
6   // 2. Validazione
7   if (!risorsa_id || quantita <= 0) {
8     return NextResponse.json({ error: "Parametri non validi" }, { status: 400 });
9   }
10
11   // 3. Autenticazione e autorizzazione
12   const userResult = await getUserParrocchia(userId);
13   const parrocchiaId = userResult.data.parrocchia_id;
14
15   // 4. Verifica risorsa appartiene alla parrocchia
16   const { data: risorsa } = await supabase
17     .from("risorsa")
18     .select("id, parrocchia_id")
19     .eq("id", risorsa_id)
20     .eq("parrocchia_id", parrocchiaId)
21     .maybeSingle();
22
23   // 5. Controllo disponibilit 
24   const { data: inventario } = await supabase
25     .from("inventario_parrocchia")
26     .select("quantita")
27     .eq("risorsa_id", risorsa_id)
28     .eq("parrocchia_id", parrocchiaId)
29     .maybeSingle();
30
31   const quantitaAssegnata = /* calcolo da assegnazioni esistenti */;
32   const quantitaDisponibile = inventario.quantita - quantitaAssegnata;
33
34   if (quantita > quantitaDisponibile) {
35     return NextResponse.json(
36       { error: `Quantita insufficiente. Disponibile: ${quantitaDisponibile}` },
37       { status: 400 }
38     );
39   }
```

```
40
41 // 6. Creazione assegnazione
42 const { data: nuovaAssegnazione } = await supabase
43   .from("assegnazione_bene")
44   .insert({
45     risorsa_id,
46     beneficiario_id,
47     quantita,
48     note,
49     data_assegnazione: new Date().toISOString()
50   })
51   .select()
52   .maybeSingle();
53
54 return NextResponse.json(nuovaAssegnazione, { status: 201 });
55 }
```

7.2.5 Fase 5: Aggiornamento UI

Dopo la risposta positiva dal server:

1. Mostra toast di successo: "Bene assegnato!"
2. Redirect a `/beni`
3. Ricarica lista beni con quantità aggiornata

7.3 Validazioni e Controlli

7.3.1 Client-Side

- Form validation (campi obbligatori)
- Verifica quantità disponibile (basata su dati cached)
- Feedback immediato all'utente

7.3.2 Server-Side

- Autenticazione utente
- Autorizzazione (verifica parrocchia)
- Validazione business logic
- Controllo disponibilità reale (query database)
- Transazioni atomiche

7.4 Gestione Errori

Esempi di errori gestiti:

```
1 // 1. Quantità insufficiente
2 if (quantita > quantitaDisponibile) {
3   return NextResponse.json(
4     { error: `Quantità insufficiente. Disponibile: ${quantitaDisponibile}`,
5       { status: 400 }
6   );
7 }
```

```
8
9 // 2. Risorsa non trovata
10 if (!risorsa) {
11     return NextResponse.json(
12         { error: "Risorsa non trovata o non appartiene alla tua parrocchia." },
13         { status: 404 }
14     );
15 }
16
17 // 3. Beneficiario non trovato
18 if (!beneficiario) {
19     return NextResponse.json(
20         { error: "Beneficiario non trovato o non appartiene alla tua parrocchia." },
21         { status: 404 }
22     );
23 }
```

8 Pattern e Convenzioni

8.1 Naming Conventions

- **Componenti:** PascalCase (DashboardLayout.tsx)
- **File API:** lowercase (route.ts)
- **Variabili:** camelCase (userData, beneficiarioId)
- **Costanti:** UPPER_SNAKE_CASE (NEXT_PUBLIC_SUPABASE_URL)

8.2 Struttura API Routes

Ogni API route esporta funzioni HTTP standard:

- **GET:** Lettura dati
- **POST:** Creazione risorse
- **PATCH:** Aggiornamento parziale
- **DELETE:** Eliminazione risorse

8.3 Gestione Stato

- **Client Components:** useState per stato locale
- **Server Components:** Props e fetch diretti
- **Global State:** Non utilizzato (preferiti fetch diretti)

8.4 Error Handling

- **API Routes:** Return NextResponse.json({ error: "..."}, { status: ... })
- **Client Components:** Try-catch con toast notifications
- **Database Errors:** Logging e messaggi user-friendly

8.5 Type Safety

- TypeScript per type checking
- Interfacce per dati API
- Type inference da Supabase queries

8.6 Styling

- Tailwind CSS utility classes
- Design system con variabili CSS
- Componenti UI riutilizzabili (shadcn/ui)

9 Conclusioni

L'architettura di KARIS è progettata per essere:

- **Scalabile:** Next.js permette crescita orizzontale
- **Manutenibile:** Separazione chiara delle responsabilità
- **Type-Safe:** TypeScript end-to-end
- **User-Friendly:** Feedback immediato e UI moderna
- **Sicura:** Autenticazione e autorizzazione a ogni livello

Il sistema utilizza tecnologie moderne e consolidate, garantendo performance, sicurezza e facilità di sviluppo.

10 Riferimenti

- Next.js Documentation: <https://nextjs.org/docs>
- Supabase Documentation: <https://supabase.com/docs>
- React Documentation: <https://react.dev>
- Tailwind CSS Documentation: <https://tailwindcss.com/docs>